

DQN Control of the Unitree G1 Left Elbow

CSCN8020 Reinforcement Learning Assignment 3. This report documents a student-written PyTorch Deep Q-Network for the approved Unitree G1 left elbow Gymnasium environment.

1. Introduction

The Unitree G1 Primer Workshop built a fixed-base MuJoCo model, a low-level proportional-derivative controller with MuJoCo bias compensation, and a Gymnasium environment for discrete high-level left elbow control. This assignment starts from that platform and replaces the transparent rule-based high-level action policy with a learned DQN policy.

The learning objective is not to redesign the robot model. The DQN only chooses among three discrete changes to the internal controller target. The approved low-level controller then applies torque and produces the physical elbow movement in simulation.

2. Environment Definition

- Environment class: `G1ElbowTargetEnv`.
- Controlled joint: `left_elbow_joint`.
- Controlled actuator: `left_elbow`.
- Observation: current angle, velocity, goal angle, and angle error.
- Actions: decrease target, hold target, or increase target.
- Training goal range: -0.8 rad to +0.8 rad.
- Episode limit: 150 environment steps.
- Success: maintain the elbow inside the tolerance for the required streak.

The reward is shaped by the absolute angle error, includes a small bonus inside the success region, and gives a terminal bonus after the required success streak. The report treats true termination as success. Time-limit truncation stops the episode but is not treated as a terminal success signal for bootstrapping.

3. DQN Method

DQN is suitable because the action space is discrete and small. The network maps a four-value continuous observation vector to three unconstrained Q-values, one for each action. The selected action is the action with the largest Q-value during greedy evaluation.

Component	Implementation
Input	4 observation values
Hidden layer 1	64 units, ReLU
Hidden layer 2	64 units, ReLU
Output	3 raw Q-values, no softmax
Loss	Huber loss
Optimizer	Adam, learning rate 0.001
Discount factor	$\gamma = 0.95$

The implementation uses an online Q-network for action-value updates and a separate target Q-network for stable Bellman targets. The target network is initialized from the online network and synchronized every 250 optimization steps.

4. Replay Buffer and Bellman Update

The replay buffer stores state, action, reward, next state, and true termination. Learning starts only after the buffer contains at least 500 transitions, which prevents very small early samples from dominating the first gradient steps. Mini-batches of 64 transitions are sampled randomly.

For each transition, the online network estimates $Q(s, a)$. The target is $r + \gamma \max_a Q_{\text{target}}(s_{\text{next}}, a)$ for non-terminal transitions. For true terminated transitions, the bootstrap term is masked out. This assignment environment can also truncate at the time limit; truncation is recorded as episode end but is not treated as a true success termination inside the replay target.

5. Training Methodology

Training was run headlessly in WSL using CPU execution. Python, NumPy, PyTorch, and the Gymnasium reset seed were controlled from the command line. The same seed policy and hyperparameters were used for both exploration-decay configurations except for the epsilon decay value.

Hyperparameter	Value
Episodes per configuration	500
Replay buffer capacity	50,000 transitions
Batch size	64
Warm-up transitions	500
Initial epsilon	1.00
Minimum epsilon	0.05
Config A epsilon decay	0.995
Config B epsilon decay	0.985
Evaluation epsilon	0.00

Both configurations finished well under the five-hour training limit. Config A took 82.0 seconds and Config B took 71.0 seconds on the tested CPU environment.

6. Exploration-Decay Results

Metric	Config A	Config B
Training episodes	500	500
Wall-clock seconds	82.0	71.0
Final epsilon	0.082	0.050
Mean reward, final 20 train episodes	14.883	15.031
Success rate, final 50 train episodes	100.0%	100.0%
Evaluation successes	20/20	20/20
Mean evaluation reward	13.282	13.167
Mean final absolute error	0.00491	0.00950

Both configurations reached 100 percent greedy evaluation success. Config B reached minimum epsilon earlier and had a slightly higher final-20 training reward. Config A produced a slightly higher mean evaluation reward and lower final absolute error, so Config A was selected as the final checkpoint.

7. Training Curves

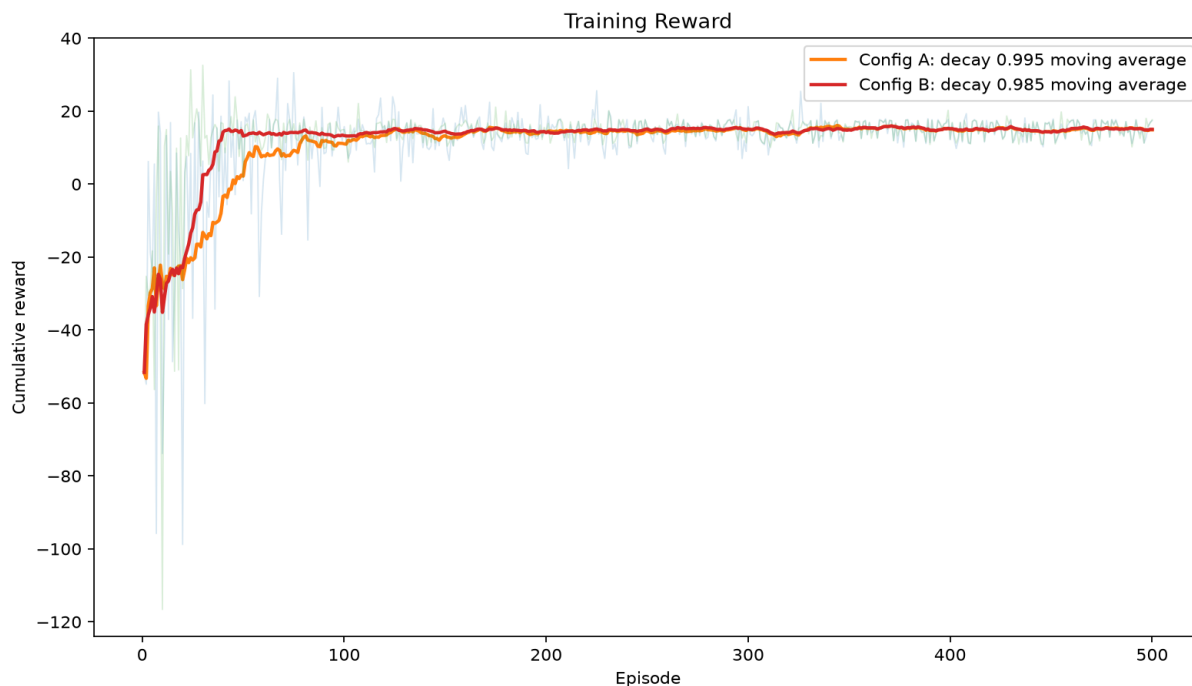


Figure. Raw training rewards are shown lightly, with a 20-episode moving average.



Figure. Training success rate using a 50-episode rolling window.

8. Exploration and Loss

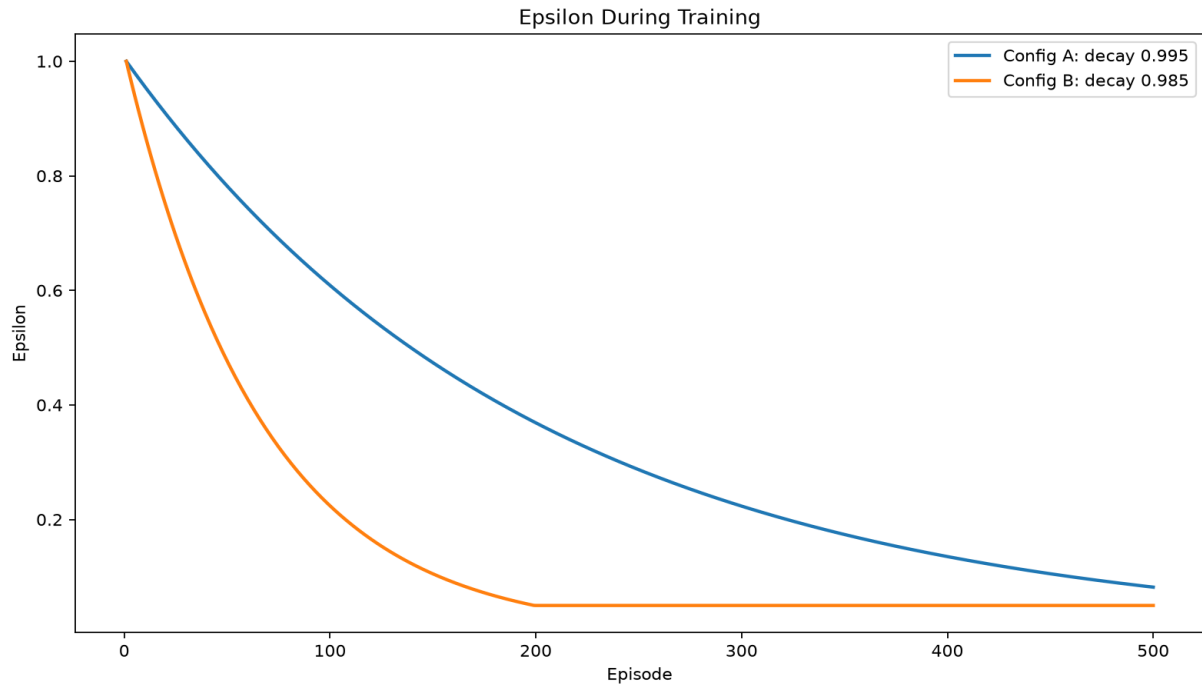


Figure. Config B decays exploration faster and reaches epsilon_min earlier.

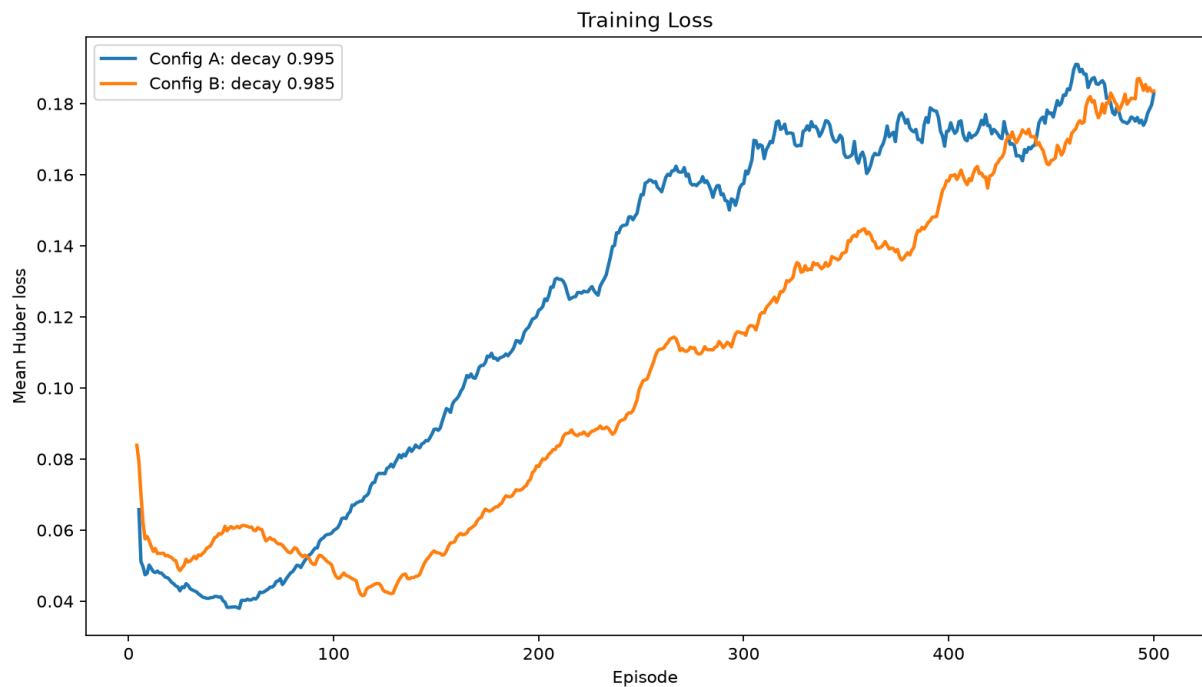


Figure. Mean Huber loss over training episodes after replay warm-up.

9. Final Greedy Evaluation

Final evaluation used epsilon = 0.0 on four required benchmark goals. Each goal was evaluated in five episodes, for 20 total episodes per configuration.

Goal	Episodes	Successes	Success rate	Mean reward
-0.8 rad	5	5/5	1.00	11.058
-0.4 rad	5	5/5	1.00	15.496
+0.4 rad	5	5/5	1.00	15.545
+0.8 rad	5	5/5	1.00	11.028
Overall	20	20/20	1.00	13.282

The selected DQN is Config A. It met the required threshold with 20 successful episodes out of 20.

Goal	Config A successes	Config B successes
-0.8 rad	5/5	5/5
-0.4 rad	5/5	5/5
+0.4 rad	5/5	5/5
+0.8 rad	5/5	5/5
Overall	20/20	20/20

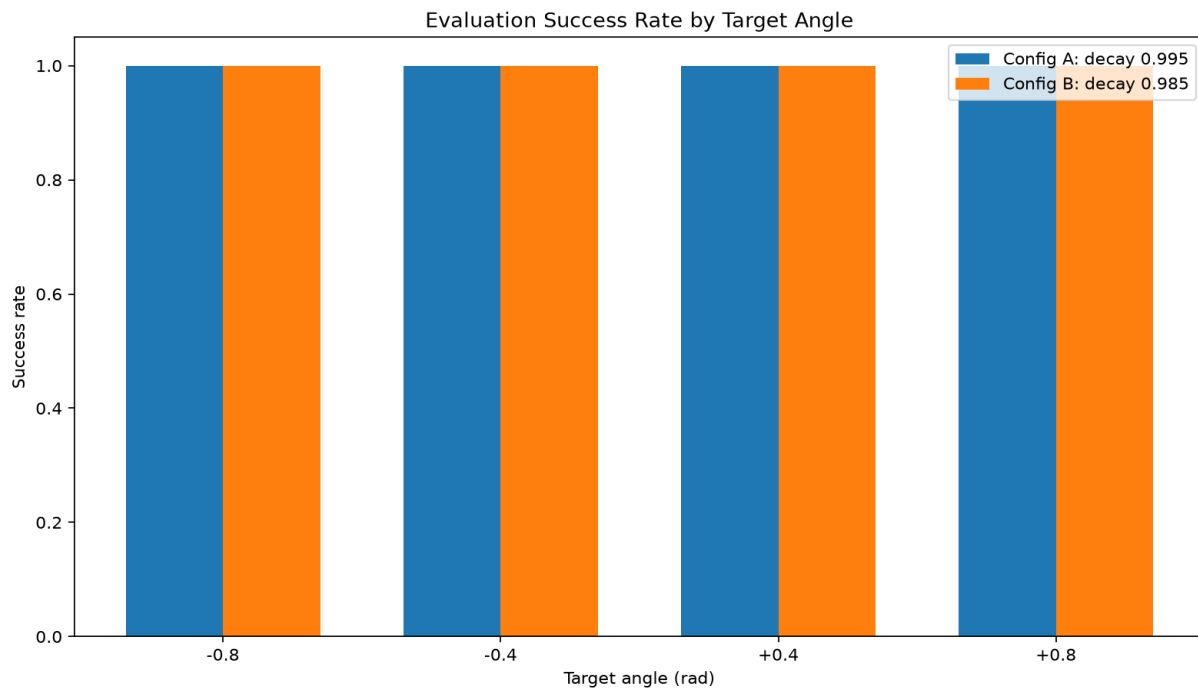


Figure. Both DQN configurations reached 100 percent success for every benchmark goal.

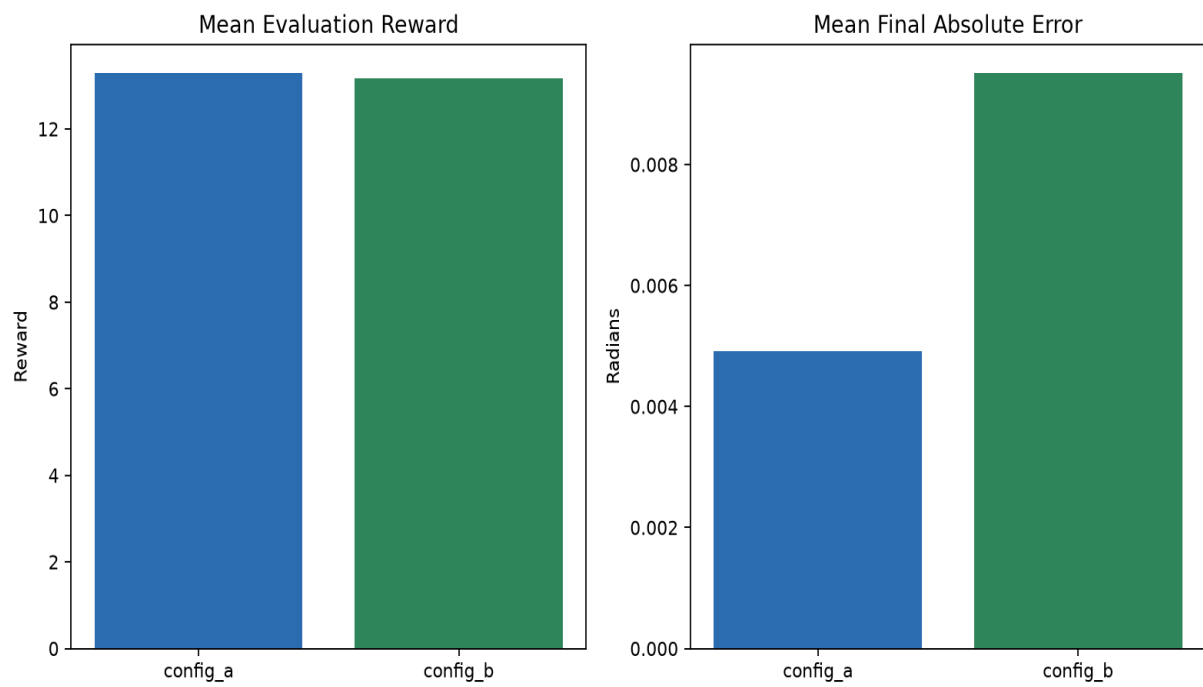


Figure. Config A had slightly better mean evaluation reward and lower final error.

10. Rule-Based Policy Comparison

Metric	Rule-based	Selected DQN
Successes / 20	20/20	20/20
Success rate	1.00	1.00
Mean reward	12.867	13.282
Mean episode length	24.0	20.0
Mean final absolute error	0.01221	0.00491
Qualitative behavior	Direct target stepping, then HOLD	Stable greedy DQN policy

The rule-based policy is more sample efficient because it uses the known structure of the environment and requires no data collection. The learned DQN needs exploration and replay updates before it matches that behavior. After training, the selected DQN generalized across all four target angles and reached the same 20/20 success rate as the rule-based baseline.

Near the goal, successful behavior requires HOLD rather than unnecessary target changes. The final DQN behavior was stable enough to complete every greedy evaluation episode, and the final absolute error was lower than the rule-based baseline in this run. A hand-written policy can still outperform a learned agent in this simple task because it directly encodes the intended target-tracking logic without exploration noise.

11. Limitations and Future Work

- Only one seed policy was used for the final controlled comparison.
- The task controls one joint, not whole-body humanoid behavior.
- The reward and success function were kept fixed as required.
- More random seeds would better estimate variability.
- A future extension could compare different target update intervals or network sizes.

12. Reproducibility and AI Disclosure

The submitted README lists exact commands for dependency setup, training, evaluation, plot generation, checkpoint loading, and rendering. AI assistance was used to help draft and organize code, tests, commands, and report text. All DQN components are included in source form and are intended to be explainable line by line.